



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Examen Parcial I: Plataforma de Triage y Citas Médicas "MedTriage Express"

Ficha Técnica del Examen

- **Fecha:** Lunes 12 de Octubre, 2026
- **Duración Máxima:** 3 horas (180 minutos)
- **Ponderación:** 20% de la nota final del curso
- **Modalidad:** Presencial (Laboratorio de Cómputo 3) - Individual
- **Motor de Base de Datos:** SQL Server Developer Edition
- **Entornos a Desarrollar:**
 - Backend: Spring Boot 3 (versión de Java disponible en su computadora) en carpeta `medtriage-backend`
 - Frontend: Angular 19 Standalone en carpeta `medtriage-frontend`

- **Restricciones:** Prohibición estricta de asistentes de IA generativa (ChatGPT, Copilot, Claude, Gemini). Prohibido el acceso a documentación externa o repositorios previos. Conexión obligatoria a GitHub con commits semánticos periódicos.

Introducción y Caso de Negocio: MedTriage Express

El centro de salud **MedTriage Express** requiere un sistema informático de pila completa (Full-Stack) para gestionar el flujo de recepción, prioridad de triaje y asignación de citas médicas de emergencia.

Usted ha sido contratado para construir desde cero tanto la **API RESTful back-end en Spring Boot** como la **interfaz cliente SPA en Angular 19 Standalone**, basándose en los scripts de base de datos SQL Server provistos por la institución.

Scripts SQL de Base de Datos (SQL Server)

Ejecute los siguientes scripts en SQL Server Management Studio (SSMS) antes de iniciar la codificación:

```
-- 1. Creacion del Esquema de Base de Datos
CREATE DATABASE MedTriageDB_<carneestudiante>;
GO

USE MedTriageDB_<carneestudiante>;
GO

CREATE TABLE Doctor (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    especialidad VARCHAR(80) NOT NULL,
    disponible BIT NOT NULL DEFAULT 1
);

CREATE TABLE Paciente (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    identificacion VARCHAR(20) NOT NULL UNIQUE,
```

```

    nombre_completo VARCHAR(120) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    telefono VARCHAR(20) NOT NULL
);

CREATE TABLE CitaMedica (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    codigo_cita VARCHAR(30) NOT NULL UNIQUE,
    paciente_id BIGINT NOT NULL,
    doctor_id BIGINT NOT NULL,
    nivel_prioridad VARCHAR(20) NOT NULL,
    motivo_consulta VARCHAR(255) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    monto_consulta DECIMAL(10,2) NOT NULL,
    fecha_cita DATETIME2 NOT NULL,
    CONSTRAINT FK_Cita_Paciente
        FOREIGN KEY (paciente_id) REFERENCES Paciente(id),
    CONSTRAINT FK_Cita_Doctor
        FOREIGN KEY (doctor_id) REFERENCES Doctor(id)
);
GO

-- 2. Datos Semilla (Seed Data)
INSERT INTO Doctor (nombre, especialidad, disponible) VALUES
('Dr. Carlos Alvarado', 'Medicina General', 1),
('Dra. Sofia Segura', 'Pediatria', 1),
('Dr. Roberto Mora', 'Urgencias', 0);

INSERT INTO Paciente (identificacion, nombre_completo, correo, telefono)
VALUES
('118230495', 'Elena Madrigal Castro', 'elena@ucr.ac.cr', '88776655'),
('207410982', 'Mario Jimenez Solano', 'mario@ucr.ac.cr', '83332211');

INSERT INTO CitaMedica
(codigo_cita, paciente_id, doctor_id, nivel_prioridad,
motivo_consulta, estado, monto_consulta, fecha_cita)
VALUES
('CIT-2026-001', 1, 1, 'ALTA',
'Fiebre alta y dificultad respiratoria', 'PENDIENTE', 25000.00, GETDATE()),
('CIT-2026-002', 2, 2, 'MEDIA',
'Dolor abdominal moderado', 'EN_ATENCION', 20000.00, GETDATE());
GO

```

Requerimientos del Examen

1. Desarrollo Back-End (`medtriage-backend` - 35%)

Cree un proyecto Spring Boot 3 en la versión de Java disponible en su computadora estructurado en las siguientes capas:

- **[] Mapeo JPA (`com.medtriage.model`):**

Implemente las entidades `Doctor.java`, `Paciente.java` y `CitaMedica.java` con las relaciones `@ManyToOne` hacia `Paciente` y `Doctor`.

[] DTOs (`com.medtriage.dto`):

- `CitaMedicaDTO`: Campos `id`, `codigoCita`, `nombrePaciente`, `nombreDoctor`, `nivelPrioridad`, `motivoConsulta`, `estado`, `montoConsulta`, `fechaCita`.

`CrearCitaDTO`: Campos `pacienteId`, `doctorId`, `nivelPrioridad` (`ALTA`, `MEDIA`, `BAJA`), `motivoConsulta`, `montoConsulta`.

[] Persistencia (`com.medtriage.repository`):

`CitaMedicaRepository`: Métodos JPA para buscar por `nivelPrioridad` y ordenar descendientemente por `fechaCita`.

[] Lógica de Negocio (`com.medtriage.service`):

`CitaMedicaService`:

- `obtenerTodas()`: Retorna lista de DTOs.
- `obtenerPorPrioridad(String prioridad)`: Retorna citas por prioridad.
- `crearCita(CrearCitaDTO dto)`: Genera código único `CIT-2026-XXXX`, valida disponibilidad del doctor, establece estado en `PENDIENTE` y guarda en la base de datos.
- `actualizarEstado(Long id, String nuevoEstado)`: Cambia el estado de la cita (`EN_ATENCION`, `ATENDIDA`, `CANCELADA`).

[] Controlador RESTful (`com.medtriage.controller`):

- Controller `@RestController` en `/api/v1/citas`.
 - Habilite CORS con `@CrossOrigin(origins = "http://localhost:4200")`.
 - Endpoints: `GET /api/v1/citas`, `GET /api/v1/citas/prioridad/{prioridad}`, `POST /api/v1/citas`, `PATCH /api/v1/citas/{id}/estado`.
-

2. Pruebas Unitarias y Manejo de Excepciones (15%)

- **[] Manejador de Excepciones Centralizado:**

Implemente `@RestControllerAdvice` en `com.medtriage.exception` retornando la norma RFC 7807 (Problem Details).

[] Prueba Unitaria con Mockito:

- En `src/test/java/com/medtriage/service/CitaMedicaServiceTest.java`, construya al menos una prueba unitaria probando el método `crearCita()` aislando el repositorio con `@Mock` y `@InjectMocks`.
-

3. Desarrollo Front-End Angular 19 (`medtriage-frontend` - 35%)

Inicialice el cliente con `ng new medtriage-frontend --standalone (CSS format, SSR No)`:

- **[] Configuración Global (`app.config.ts` y `environment.ts`):**
- Registre `provideHttpClient(withFetch())` en `app.config.ts`.

Defina `API_URL: 'http://localhost:8080/api/v1/'` en `environment.ts`.

[] Modelos e Interfaz (`src/app/models/cita.model.ts`):

Interfaces `CitaMedica`, `Doctor`, `Paciente` y `CrearCitaPayload`.

[] Servicio Angular (`src/app/services/cita.service.ts`):

- Inyección mediante `inject(HttpClient)`.

Métodos `getCitas()`, `getCitasPorPrioridad(prioridad)`, `crearCita(payload)`, `actualizarEstado(id, estado)`.

[] Componente Dashboard con Angular Signals (`CitaListComponent`):

- **Ruta:** `/citas`
- Uso de Signals (`signal()`, `computed()`) para almacenar la lista de citas y aplicar un filtro reactivo por prioridad (TODAS, ALTA, MEDIA, BAJA).

Tabla responsiva con badges de color por estado y botón interactivo para avanzar el estado de la cita.

[] Componente Formulario de Triage (`CitaFormComponent`):

- **Ruta:** `/nueva-cita`
- Formulario con `[(ngModel)]` para registrar paciente, doctor, prioridad, motivo y monto.

Procesamiento y envío mediante `CitaService`.

[] Navegación y Rutas (`app.routes.ts`):

- Enrutamiento entre `/citas` y `/nueva-cita` con barra de navegación superior en `app.component.html`.

4. Flujo de Trabajo en Git y Commits Semánticos (15%)

- [] Repositorio inicializado con `.gitignore` adecuado.
- [] Mínimo 5 commits semánticos individuales durante el desarrollo de la prueba (ejemplos: `feat(backend): add JPA entities`, `feat(angular): implement CitaListComponent with Signals`).

Distribución Recomendada del Tiempo (180 Minutos)

Bloque de Tiempo	Fase de Desarrollo	Duración
Bloque 1	Ejecutar script SQL Server y crear proyectos base	20 min
Bloque 2	Desarrollo de Backend (Entidades, DTOs, Service, Controller)	50 min
Bloque 3	Manejo de Excepciones RFC 7807 y Prueba Unitaria JUnit/Mockito	20 min
Bloque 4	Setup de Angular 19, <code>environment.ts</code> , Models y <code>CitaService</code>	20 min
Bloque 5	Componentes Standalone (<code>CitaListComponent</code> con Signals y <code>CitaFormComponent</code>)	50 min
Bloque 6	Integración Full-Stack, verificación final y Commits en GitHub	20 min

Rúbrica de Evaluación (100 Puntos)

Criterio	Puntos	Descripción
----------	--------	-------------

Arquitectura Back-End RESTful	35 pts	Mapeo JPA correcto en SQL Server, capas DTO/Service/Controller, validaciones de negocio y política CORS habilitada.
Front-End Angular 19 SPA	35 pts	Componentes Standalone enrutados, reactividad con Angular Signals (<code>signal()</code> , <code>computed()</code>), consumo HTTP y Data Binding.
Excepciones & Pruebas Unitarias	15 pts	Formateo RFC 7807 con <code>@RestControllerAdvice</code> y prueba unitaria en Mockito ejecutando exitosamente.
Git & Versionamiento	15 pts	Commits semánticos periódicos durante el examen en la cuenta personal de GitHub del estudiante.